Gamified Learning: Research and Suggested Improvements for Cipher Valley

Team T334 “Avengers” · DSCPP-T334 · Phase-II Project Progress
Pending task 3: research gamified learning approaches and suggest improvements to learning activities
Prepared by Bhavishya Bhardwaj (Team Member 3) · 22 October 2026

In short

- **Gamification helps learning on average,** but only when it is designed well.
- **What works:** a story, visible progress, quick feedback, teamwork and easy practice formats.
- **What backfires:** points, badges and public leaderboards added on top of an ordinary activity.
- **Our best opportunity:** make the data structures the game itself. The quest log becomes a linked list and the item lockers become a hash table.
- Six suggestions follow, then a simple roadmap.

1. What the research says

- **It helps on average.** Three recent meta-analyses found overall gains between g ≈ 0.46 and 0.82 (g is an effect size: 0.2 is small, 0.5 medium, 0.8 large) (Huang et al., 2020; Bai et al., 2020; Li et al., 2023). Gains in knowledge are more reliable than gains in motivation or behaviour (Sailer & Homner, 2020).
- **Design matters.** A story or game world, and a mix of teamwork and competition, worked best (Sailer & Homner, 2020). In programming courses, results depend on which elements are used and on whether learners can shape their own path (Ishaq et al., 2024).
- **Leaderboards and badges can backfire.** In a 16-week course, the version with a leaderboard and badges left students less motivated, with lower exam scores, than the same course without them (Hanus & Fox, 2015).
- **Code puzzles help beginners.** Parsons problems, where you put mixed-up code blocks in order, are solved faster than writing code from scratch with the same learning gain (Ericson et al., 2023).
- **Games built around a data structure look promising.** DS-Hacker is an adventure game built around a binary search tree (Rojas-Salazar et al., 2020). Muse teaches arrays and linked lists; in a small test of 27 students the game group scored 81.5% against 72% (Sharma et al.).

Many of these studies are small or short, so treat them as guidance rather than proof.

2. Where Cipher Valley stands

The Phase-II report shows a Phaser.js campus with movement, collision and an E-key lab interaction, plus C modules for a linked-list quest manager and a linear-probing hash table (stack and queue are in progress). The game and the C code are still separate, and the report does not yet describe what a player actually learns inside the game. The suggestions below add that learning layer.

3. Suggested improvements

1. Make the data structures the game itself

- **Quest Board = linked list.** Quests appear as connected nodes. Accepting a quest adds a node and completing it removes one. This reuses your C quest manager.
- **Item lockers = hash table.** Items go into numbered lockers using a simple hash. If two items collide, the game shows linear probing moving one to the next free locker. This reuses your hash table.
- **Stack and queue (ideas):** a cafeteria line for a queue, a pile of lab manuals for a stack.

Abstract ideas are easier to learn when the player can see them.

2. Use a four-step loop at the lab

1. **Learn:** a short worked example.
2. **Arrange:** put mixed-up code blocks in order.
3. **Fix:** find the bug in a short C snippet.
4. **Write:** fill in or write a small function.

Your own roadblocks (pointer mistakes, hash collisions) make ready-made “Fix” puzzles. Let experienced players skip ahead.

3. Give instant feedback and hints

After each attempt, say what went wrong and why. Offer three hints: a reminder, then the line to look at, then a worked example, with no penalty for using them. Feedback that builds a sense of progress supports motivation, while competition and controlling rewards can reduce it (arXiv:2203.16175). Start with simple answer-checking in JavaScript.

4. Progress by mastery

Give each concept three stages: Explore, Practise, Master. Unlock the next area when the player shows mastery. If someone keeps succeeding, offer a harder version; if they struggle, offer an easier one or an extra example. Let players change the difficulty themselves.

5. Reward progress and add a story

- **Rewards:** a progress map, mastery badges (“List Keeper”, “Collision Cracker”) and story unlocks. Avoid points for every click and avoid public leaderboards. For comparison, use team goals.
- **Story:** for example, a mysterious “Cipher” has scrambled the campus systems and each solved concept restores a building. Keep dialogue short and skippable.

6. Connect the game to the C code

This also answers Challenge 5 in your report. Two options:

- **Option A (try first):** compile the C modules to WebAssembly with Emscripten and call them from JavaScript with ccall or cwrap (MDN Web Docs).
- **Option B:** rebuild the structures in JavaScript for the visuals and keep the C code as the reference.

Both run your own code. Letting players write and compile their own C in the game needs a browser compiler or a server, so for now use puzzles the game can check (arrange, fix, fill in).

4. Simple roadmap

When	What to do
Near term	Write one learning goal for each quest. Turn your own bugs into “Fix” puzzles. Add a progress indicator and a mastery badge in the lab.
Next phase	Build arrange-the-code puzzles for the linked list and hash table. Add feedback messages and the hint steps. Drive the quest board and lockers from the DSA code.
Later	Adaptive difficulty and revision quests that bring old topics back after a few days. Team challenges.

Suggested first step: build one complete loop for the linked list (learn, arrange, fix, write, with hints and a badge), then reuse it for the hash table.

5. How to test it

- Try it with 8–15 classmates, in at least two sessions on different days.
- Give a short quiz (5–8 questions) before and after each module.
- Note attempts, hints used and time taken, plus a few questions on enjoyment and difficulty.

With a group this small, the results are only indicative.

References

Bai, S., Hew, K. F., & Huang, B. (2020). Does gamification improve student learning outcome? Evidence from a meta-analysis and synthesis of qualitative data in educational contexts. *Educational Research Review*. https://www.gbl.uzh.ch/quartz/references/Bai-et-al.-(2020)

Ericson, B. J., Pearce, J. L., Rodger, S. H., et al. (2023). Conducting multi-institutional studies of Parsons problems. *ITiCSE 2023*, 571–572. https://researchportal.hw.ac.uk/en/publications/conducting-multi-institutional-studies-of-parsons-problems/

Hanus, M. D., & Fox, J. (2015). Assessing the effects of gamification in the classroom: A longitudinal study on intrinsic motivation, social comparison, satisfaction, effort, and academic performance. *Computers & Education, 80*, 152–161.

Huang, R., Ritzhaupt, A. D., Sommer, M., et al. (2020). The impact of gamification in educational settings on student learning outcomes: A meta-analysis. *Educational Technology Research and Development*. https://pure.ecnu.edu.cn/en/publications/the-impact-of-gamification-in-educational-settings-on-student-lea/

Ishaq, K., Alvi, A., Ikram ul Haq, M., et al. (2024). Level up your coding: A systematic review of personalized, cognitive, and gamified learning in programming education. *PeerJ Computer Science, 10*, e2310. https://doi.org/10.7717/peerj-cs.2310

Li, M., Ma, S., & Shi, Y. (2023). Examining the effectiveness of gamification as a tool promoting teaching and learning in educational settings: A meta-analysis. *Frontiers in Psychology, 14*. https://doi.org/10.3389/fpsyg.2023.1253549

MDN Web Docs. Compiling a new C/C++ module to WebAssembly. https://developer.mozilla.org/en-US/docs/WebAssembly/C_to_Wasm

Rojas-Salazar, A., Ramírez-Alfaro, P., & Haahr, M. (2020). Learning binary search trees through serious games. *OASIcs, ICPEC 2020*. https://drops.dagstuhl.de/opus/volltexte/2020/12309/

Sailer, M., & Homner, L. (2020). The gamification of learning: A meta-analysis. *Educational Psychology Review, 32*(1), 77–112. https://opus.bibliothek.uni-augsburg.de/opus4/frontdoor/index/index/docId/109056

Sharma, V., Musarrat, R., Chimalakonda, S., & Reddy, Y. R. Muse: A musically inspired game to teach arrays and linked lists. *ICCE Proceedings (APSCE)*. https://library.apsce.net/index.php/ICCE/article/view/2344

When gamification spoils your learning: A qualitative case study of gamification misuse in a language-learning app. arXiv:2203.16175. https://arxiv.org/pdf/2203.16175